Assignment-5

Medina Alish, SE-2538

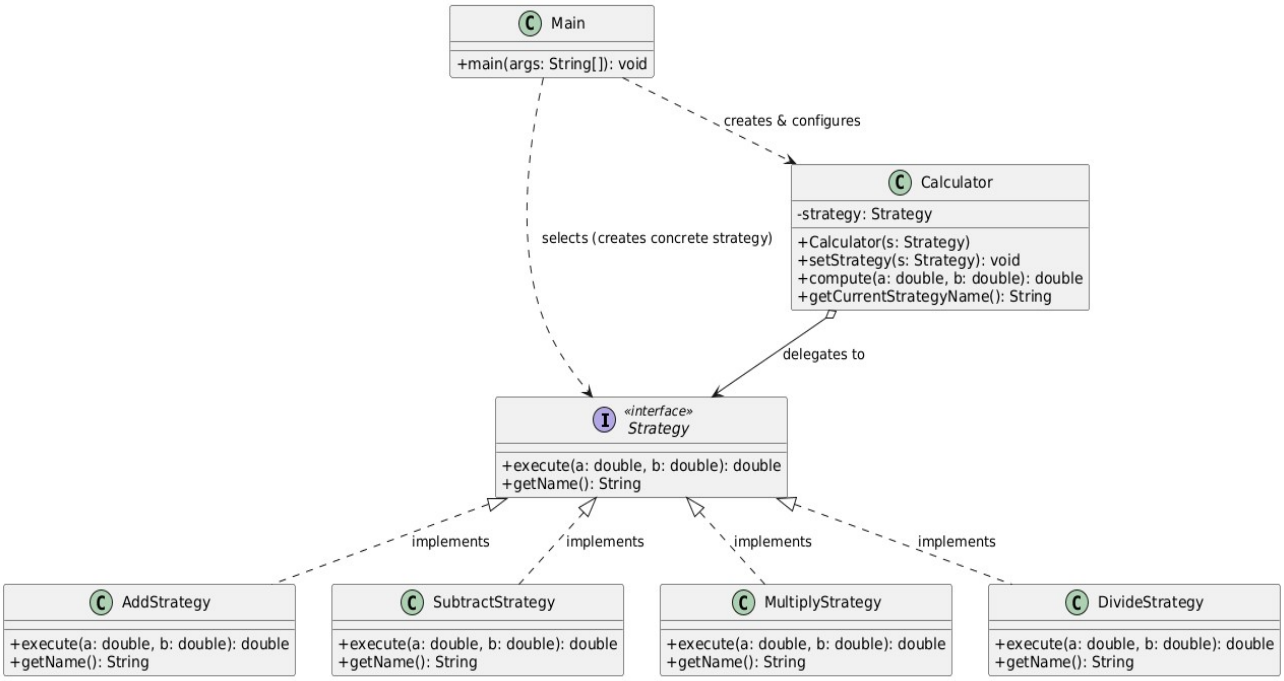
Repository Link: aalishmee/sdp5

1. Introduction

For this assignment, I implemented the Strategy pattern using the Calculator domain (Option B). A calculator performs binary arithmetic operations - addition, subtraction, multiplication, and division. Without Strategy, a single Calculator class would need a chain of if/else or switch statements to select the algorithm. Every new operation would require modifying that class, violating the Open/Closed Principle and making the code harder to test and extend.

The Strategy pattern solves this by turning each arithmetic operation into its own class that implements a common Strategy interface. The Calculator (Context) holds a reference to the interface and delegates the computation to whichever strategy is currently set. The client (Main) chooses the strategy at runtime and can switch it freely. The context never contains a single if or switch about which operation to perform - the algorithm selection lives entirely in the client, and the arithmetic logic lives entirely in the strategies.

2. UML Class Diagram



3. Clean Code Principles

I identified and applied the following five Clean Code principles in my implementation:

1. Elimination of Conditional Logic in Favor of Polymorphism

The Context has no if or switch on strategy type. The computation is delegated through the interface.

Before (Bad):

```
public double compute(String op, double a, double b) {
    if (op.equals("add"))      return a + b;
    else if (op.equals("sub")) return a - b;
    else if (op.equals("mul")) return a * b;
    else if (op.equals("div")) return a / b;
    throw new IllegalArgumentException("Unknown op");
}
```

After (Good):

```
public double compute(double a, double b) {
    return strategy.execute(a, b);
}
```

2. Meaningful, Intention-Revealing Names

Each strategy class is named after exactly what it does: AddStrategy, SubtractStrategy, MultiplyStrategy, DivideStrategy. A reader seeing new AddStrategy() immediately knows the behavior. Names like Op1 or Handler would force the reader to open the file.

3. Single Responsibility Principle (SRP)

Each strategy class contains only the one algorithm it represents - nothing else. Parsing lives in Main, arithmetic lives in the strategies, orchestration lives in Calculator. Before (Bad): a single Calculator class validating input, selecting the algorithm, computing, and formatting the output. After (Good): each of those concerns is in its own class.

4. Context Remains Algorithm-Agnostic

Calculator only knows the Strategy interface. It never imports or references AddStrategy, DivideStrategy, or any other concrete class. This means the context can be compiled and tested without any concrete strategy existing.

Before (Bad):

```
if (op.equals("add")) new AddStrategy().execute(a, b);
```

After (Good):

```
private Strategy strategy;
public void setStrategy(Strategy strategy) { this.strategy = strategy; }
public double compute(double a, double b) { return strategy.execute(a, b); }
```

5. Easy Extensibility (Open/Closed Principle)

Adding a new operation requires only two things: a new class implementing Strategy, and one line in the client. No existing file is modified.

```
public class PowerStrategy implements Strategy {
    public double execute(double a, double b) { return Math.pow(a, b); }
    public String getName() { return "Power"; }
}
```

```
case "pow" -> calculator.setStrategy(new PowerStrategy());
```

Calculator, Strategy, and all existing strategies remain untouched.

4. Strategy vs. State

Strategy and State share the same structural shape - a Context holding a reference to an interface, with concrete implementations behind it. The key difference is intent and who triggers transitions.

In the State pattern, the concrete state objects cause the Context to transition into a new state as a side effect of their behavior. The client does not choose the next state - the state machine drives itself.

In the Strategy pattern, the client explicitly chooses which algorithm the Context should use, and the algorithm does not trigger any transition. In my calculator, choosing AddStrategy never causes a switch to MultiplyStrategy. The user switches freely at any time via setStrategy().

My problem therefore calls for Strategy: arithmetic operations are independent, interchangeable algorithms selected externally - not states of a self-transitioning object.

5. Conclusion

The Strategy pattern successfully removes the conditional logic that would otherwise live inside the Calculator. It gives the client a clean API (setStrategy + compute) and makes each algorithm independently testable.

A defining property of this pattern is that the Context stays closed to modification - it can be compiled and shipped before any concrete strategy exists. However, the trade-off is one new class per algorithm. For four trivial operations this may seem like over-engineering, but the payoff becomes clear as the algorithm family grows in complexity or size - scientific operations, financial calculations, unit conversions, and so on.